Chapter 3 described how to organize data into computer memory using an array and how to perform basic operations on that data. Chapter 5 then described how to organize data using linked lists. The most important operation performed on a list is the search algorithm. Using the search algorithm, you can do the following:

- Determine whether a particular item is in the list.
- If the data is specially organized (for example, sorted), find the location in the list where a new item can be inserted.
- Find the location of an item to be deleted.

The search algorithm's performance, therefore, is crucial. If the search is slow, it takes a large amount of computer time to accomplish your task; if the search is fast, you can accomplish your task quickly.

Search Algorithms

Chapters 3 and 5 described how to implement the sequential search algorithm. This chapter discusses other search algorithms and analyzes them. The analysis of algorithms enables programmers to decide which algorithm to use for a specific application. Before describing these algorithms, let us make the following observations.

Associated with each item in a data set is a special member that uniquely identifies the item in the data set. For example, if you have a data set consisting of student records, then the student ID uniquely identifies each student in a particular school. This unique member of the item is called the **key** of the item. The keys of the items in the data set are used in such operations as searching, sorting, insertion, and deletion. For instance, when we search the data set for a particular item, we compare the key of the item for which we are searching with the keys of the items in the data set.

As previously remarked, in addition to describing searching algorithms, this chapter analyzes these algorithms. In the analysis of an algorithm, the key comparisons refer to comparing the key of the search item with the key of an item in the list. Moreover, the number of key comparisons refers to the number of times the key of the item (in algorithms such as searching and sorting) is compared with the keys of the items in the list.

In Chapter 3, we designed and implemented the `class arrayListType` to implement a list and the basic operations in an array. Because this chapter refers to this class, for easy reference we give its definition, without documentation to save space, here:

```
template <class elemType>
class arrayListType
{
public:
    const arrayListType<elemType>& operator=
                          (const arrayListType<elemType>&);
```

```
    bool isEmpty() const;
    bool isFull() const;
    int listSize() const;
    int maxListSize() const;
    void print() const;
    bool isItemAtEqual(int location, const elemType& item) const;
    void insertAt(int location, const elemType& insertItem);
    void insertEnd(const elemType& insertItem);
    void removeAt(int location);
    void retrieveAt(int location, elemType& retItem) const;
    void replaceAt(int location, const elemType& repItem);
    void clearList();
    int seqSearch(const elemType& item) const;
    void insert(const elemType& insertItem);
    void remove(const elemType& removeItem);

    arrayListType(int size = 100);

    arrayListType(const arrayListType<elemType>& otherList);

    ~arrayListType();

protected:
    elemType *list;  //array to hold the list elements
    int length;      //to store the length of the list
    int maxSize;     //to store the maximum size of the list
};
```


Sequential Search

The sequential search (also called linear search) on array-based lists was described in Chapter 3, and the sequential search on linked lists was covered in Chapter 5. The sequential search works the same for both array-based and linked lists. The search always starts at the first element in the list and continues until either the item is found in the list or the entire list is searched.

Because we are interested in the performance of the sequential search (that is, the analysis of this type of search), for easy reference and for the sake of completeness, we give the sequential search algorithm for array-based lists (as described in Chapter 3). If the search item is found, its index (that is, its location in the array) is returned. If the search is unsuccessful, `-1` is returned. Note that the following sequential search does not require the list elements to be in any particular order.

```
template <class elemType>
int arrayListType<elemType>::seqSearch(const elemType& item) const
{
    int loc;
    bool found = false;

    for (loc = 0; loc < length; loc++)
        if (list[loc] == item)
```

```
        {
            found = true;
            break;
        }

    if (found)
        return loc;
    else
        return -1;
} //end seqSearch
```

NOTE You can also write a recursive algorithm to implement the sequential search algorithm. (See Programming Exercise 1 at the end of this chapter.)

SEQUENTIAL SEARCH ANALYSIS

This section analyzes the performance of the sequential search algorithm in both the worst case and the average case.

The statements before and after the loop are executed only once and, hence, require very little computer time. The statements in the `for` loop are the ones that are repeated several times. For each iteration of the loop, the search item is compared with an element in the list, and a few other statements are executed, including some other comparisons. Clearly, the loop terminates as soon as the search item is found in the list. Therefore, the execution of the other statements in the loop is directly related to the outcome of the key comparison. Also, different programmers might implement the same algorithm differently, although the number of key comparisons would typically be the same. The speed of a computer can also easily affect the time an algorithm takes to perform, but not the number of key comparisons.

Therefore, when analyzing a search algorithm, we count the number of key comparisons because this number gives us the most useful information. Furthermore, the criteria for counting the number of key comparisons can be applied equally well to other search algorithms.

Suppose that L is list of length n. We want to determine the number of key comparisons made by the sequential search when the L is searched for a given item.

If the search item is not in the list, we then compare the search item with every element in the list, making n comparisons. This is an unsuccessful case.

Suppose that the search item is in the list. Then the number of key comparisons depends on where in the list the search item is located. If the search item is the first element of L, we make only one key comparison. This is the best case. On the other hand, if the search item is the last element in the list, the algorithm makes n comparisons. This is the worst case. The best and worst cases are not likely to occur every time we apply the sequential search on L, so it would be more helpful if we could determine the average behavior of the algorithm. That is, we need to determine the average number of key comparisons the sequential search algorithm makes in the successful case.

To determine the average number of comparisons in the successful case of the sequential search algorithm:

1. Consider all possible cases.
2. Find the number of comparisons for each case.
3. Add the number of comparisons and divide by the number of cases.

If the search item, called the **target**, is the first element in the list, one comparison is required. If the target is the second element in the list, two comparisons are required. Similarly, if the target is the *k*th element in the list, *k* comparisons are required. We assume that the target can be any element in the list; that is, all list elements are equally likely to be the target. Suppose that there are *n* elements in the list. The following expression gives the average number of comparisons:

$$\frac{1 + 2 + \ldots + n}{n}$$

It is known that

$$1 + 2 + \cdots + n = \frac{n(n+1)}{2}$$

Therefore, the following expression gives the average number of comparisons made by the sequential search in the successful case:

$$\frac{1 + 2 + \ldots + n}{n} = \frac{1}{n}\frac{n(n+1)}{2} = \frac{n+1}{2}$$

This expression shows that, on average, the sequential search searches half the list. It, thus, follows that if the list size is 1,000,000, on average, the sequential search makes 500,000 comparisons. As a result, the sequential search is not efficient for large lists.

Ordered Lists

A list is ordered if its elements are ordered according to some criteria. The elements of a list are usually in ascending order. Several operations that can be performed on an ordered list are similar to the operations performed on an arbitrary list. For example, determining whether the list is empty or full, determining the length of the list, printing the list, and clearing the list for an ordered list are the same operations as those on an unordered list. Therefore, to define an ordered list as an abstract data type (ADT), by using the mechanism of inheritance, we can derive the class to implement the ordered lists from the `class arrayListType` discussed in the previous section. Depending on whether a specific application of a list can be stored in either an array or a linked list, we define two classes.

The following `class`, `orderedArrayListType`, defines an ordered list stored in an array as an ADT:

```
template <class elemType>
class orderedArrayListType: public arrayListType<elemType>
{
public:
    orderedArrayListType(int size = 100);
      //constructor

   ...
      //We will add the necessary members as needed.

private:
    //We will add the necessary members as needed.
}
```

Chapter 5 defined the following class to implement ordered linked lists:

```
template <class elemType>
class orderedLinkedListType: public linkedListType<elemType>
{
public:
    ...
}
```

Binary Search

As you can see, the sequential search is not efficient for large lists because, on average, the sequential search searches half the list. We therefore describe another search algorithm, called the binary search, which is very fast. However, a binary search can be performed only on ordered lists. We, therefore, assume that the list is ordered. In the next chapter, we describe several sorting algorithms.

The binary search algorithm uses the divide-and-conquer technique to search the list. First, the search item is compared with the middle element of the list. If the search item is found, the search terminates. If the search item is less than the middle element of the list, we restrict the search to the first half of the list; otherwise, we search the second half of the list.

Consider the sorted list of `length = 12` in Figure 9-1.

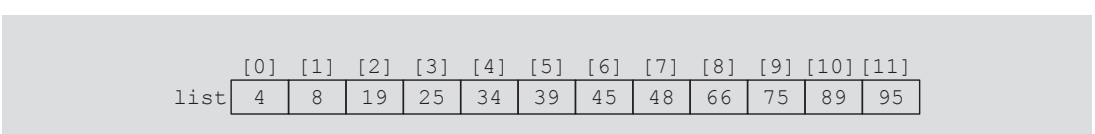

FIGURE 9-1 List of length 12

Suppose that we want to determine whether 75 is in the list. Initially, the entire list is the search list (see Figure 9-2).

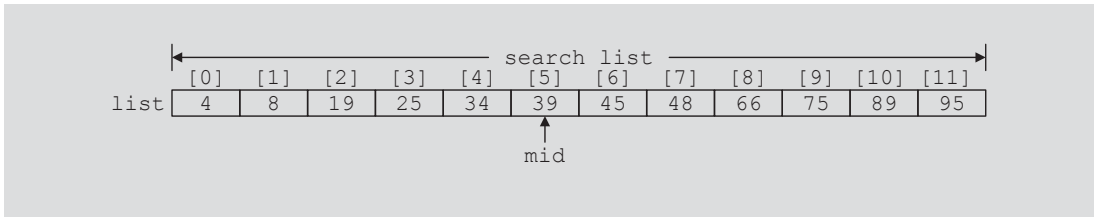

FIGURE 9-2 Search list, `list[0]...list[11]`

First, we compare 75 with the middle element in this list, `list[5]` (which is 39). Because `75 ≠ list[5]` and `75 > list[5]`, we restrict our search to the list `list[6]...list[11]`, as shown in Figure 9-3.

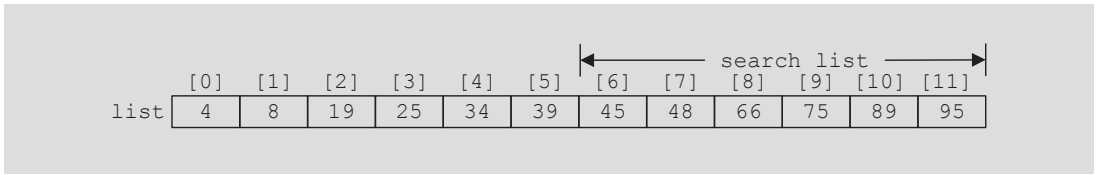

FIGURE 9-3 Search list, `list[6]...list[11]`

This process is now repeated on the list `list[6]...list[11]`, which is a list of `length = 6`.

Because we need to determine the middle element of the list frequently, the binary search algorithm is typically implemented for array-based lists. To determine the middle element of the list, we add the starting index, `first`, and the ending index, `last`, of the search list and then divide by 2 to calculate its index. That is, `mid = (first + last) / 2`.

Initially, `first = 0` and `last = length - 1` (this is because an array index in C++ starts at 0 and `length` denotes the number of elements in the list).

The following C++ function implements the binary search algorithm. If the item is found in the list, its location is returned; if the search item is not in the list, `-1` is returned.

```
template<class elemType>
int orderedArrayListType<elemType>::binarySearch
                                          (const elemType& item) const
{
    int first = 0;
    int last = length - 1;
    int mid;

    bool found = false;
```

```
    while (first <= last && !found)
    {
        mid = (first + last) / 2;

        if (list[mid] == item)
            found = true;
        else if (list[mid] > item)
            last = mid - 1;
        else
            first = mid + 1;
    }

    if (found)
        return mid;
    else
        return -1;
}//end binarySearch
```

In the binary search algorithm, each time through the loop we make two key comparisons. The only exception is in the successful case; the last time through the loop only one key comparison is made.

NOTE

The binary search algorithm, as given in this chapter, uses an iterative control structure (the `while` loop) to compare the search item with the list elements. You can also write a recursive algorithm to implement the binary search algorithm. (See Programming Exercise 2 at the end of this chapter.)

Example 9-1 further illustrates how the binary search algorithm works.

EXAMPLE 9-1

Consider the list given in Figure 9-4.

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]	[11]
list	4	8	19	25	34	39	45	48	66	75	89	95

FIGURE 9-4 Sorted list for a binary search

The number of elements in this list is `12`, so `length = 12`. Suppose that we are searching for item `89`. Table 9-1 shows the values of `first`, `last`, and `mid` each time through the loop. It also shows the number of times the item is compared with an element in the list each time through the loop.

TABLE 9-1 Values of `first`, `last`, and `mid` and the number of comparisons for search item 89

Iteration	first	last	Mid	list[mid]	Number of comparisons
1	0	11	5	39	2
2	6	11	8	66	2
3	9	11	10	89	1 (found is true)

The item is found at `location 10`, and the total number of comparisons is 5.

Next, let us search the list for item `34`. Table 9-2 shows the values of `first`, `last`, and `mid` each time through the loop. It also shows the number of times the item is compared with an element in the list each time through the loop.

TABLE 9-2 Values of `first`, `last`, and `mid` and the number of comparisons for search item 34

Iteration	first	last	mid	list[mid]	Number of comparisons
1	0	11	5	39	2
2	0	4	2	19	2
3	3	4	3	25	2
4	4	4	4	34	1 (found is true)

The item is found at `location 4`, and the total number of comparisons is 7.

Let us now search for item `22`, as shown in Table 9-3.

TABLE 9-3 Values of `first`, `last`, and `mid` and the number of comparisons for search item 22

Iteration	first	last	mid	list[mid]	Number of comparisons
1	0	11	5	39	2
2	0	4	2	19	2
3	3	4	3	25	2
4	3	2	The loop stops (because first > last)		

This is an unsuccessful search. The total number of comparisons is 6.

PERFORMANCE OF BINARY SEARCH

Suppose that L is a sorted list of size 1024 and we want to determine if an item x is in L. From the binary search algorithm, it follows that every iteration of the **while** loop cuts the size of the search list by half. (For example, see Figures 9-2 and 9-3.) Because $1024 = 2^{10}$, the **while** loop will have, at most, 11 iterations to determine whether x is in L. Because every iteration of the **while** loop makes two item (key) comparisons, that is, x is compared twice with the elements of L, the binary search will make, at most, 22 comparisons to determine whether x is in L. On the other hand, recall that a sequential search on average will make 512 comparisons to determine whether x is in L.

To better understand how fast binary search is compared with sequential search, suppose that L is of size 1048576. Because $1048576 = 2^{20}$, it follows that the **while** loop in a binary search will have at most 21 iterations to determine whether an element is in L. Every iteration of the **while** loop makes two key (that is, item) comparisons. Therefore, to determine whether an element is in L, a binary search makes at most 42 item comparisons.

Note that $40 = 2 \star 20 = 2 \star \log_2 2^{20} = 2 \star \log_2(1048576)$.

In general, suppose that L is a sorted list of size n. Moreover, suppose that n is a power of 2, that is, $n = 2^m$, for some nonnegative integer m. After each iteration of the **for** loop, about half the elements are left to search, that is, the search sublist for the next iteration is half the size of the current sublist. For example, after the *first* iteration, the search sublist is of size about $n/2 = 2^{m-1}$. It is easy to see that the maximum number of the iteration of the **for** loop is about $m + 1$. Also $m = \log_2 n$. Each iteration makes 2 key comparisons. Thus, the maximum number of comparisons to determine whether an element x is in L is $2(m + 1) = 2(\log_2 n + 1) = 2\log_2 n + 2$.

In the case of a successful search, it can be shown that for a list of length n, on average, a binary search makes $2\log_2 n - 3$ key comparisons. In the case of an unsuccessful search, it can be shown that for a list of length n, a binary search makes approximately $2\log_2(n+1)$ key comparisons.

Now that we know how to effectively search an ordered list stored in an array, let us discuss how to insert an item into an ordered list.

Insertion into an Ordered List

Suppose that you have an ordered list and want to insert an item in the list. After insertion, the resulting list must also be ordered. Chapter 5 described how to insert an item into an ordered linked list. This section describes how to insert an item into an ordered list stored in an array.

To store the item in the ordered list, first we must find the place in the list where the item is to be inserted. Then we slide the list elements one array position down to make room for the item to be inserted, and then we insert the item. Because the list is sorted and stored in an array, we can use an algorithm similar to the binary search algorithm to find the place in the list where the item is to be inserted. We can then use the function

`insertAt` (of the `class arrayListType`) to insert the item. (Note that we cannot use the binary search algorithm as designed previously because it returns `-1` if the item is not in the list. Of course, we can write another function using the binary search technique to find the position in the array where the item is to be inserted.) Therefore, the algorithm to insert the item is: (The special cases, such as inserting an item in an empty list or in a full list, are handled separately.)

1. Use an algorithm similar to the binary search algorithm to find the place where the item is to be inserted.
2. `if` the item is already in this list
   `output` an appropriate message
   `else`
   use the function `insertAt` to insert the item in the list.

The following function, `insertOrd`, implements this algorithm.

```
template <class elemType>
void orderedArrayListType<elemType>::insertOrd(const elemType& item)
{
    int first = 0;
    int last = length - 1;
    int mid;

    bool found = false;

    if (length == 0)   //the list is empty
    {
        list[0] = item;
        length++;
    }
    else if (length == maxSize)
        cerr << "Cannot insert into a full list." << endl;
    else
    {
        while (first <= last && !found)
        {
            mid = (first + last) / 2;

            if (list[mid] == item)
                found = true;
            else if (list[mid] > item)
                last = mid - 1;
            else
                first = mid + 1;
        }//end while

        if (found)
            cerr << "The insert item is already in the list. "
                 << "Duplicates are not allowed." << endl;
        else
```

```
        {
            if (list[mid] < item)
                mid++;

            insertAt(mid, item);
        }
    }
}//end insertOrd
```

Similarly, you can write a function to remove an element from an ordered list; see Programming Exercise 6 at the end of this chapter.

If we add the binary search algorithm and the `insertOrd` algorithm to the `class orderedArrayListType`, the definition of this class is as follows:

```
template <class elemType>
class orderedArrayListType: public arrayListType<elemType>
{
public:
    void insertOrd(const elemType&);
    int binarySearch(const elemType& item) const;
    orderedArrayListType(int size = 100);
};
```

Because the `class orderedArrayListType` is derived from the `class arrayListType`, and the list elements of an `orderedArrayListType` are ordered, we must override the functions `insertAt` and `insertEnd` of the `class arrayListType` in the `class orderedArrayListType`. We do this so that if these functions are used by an object of type `orderedArrayListType`, then after using these functions, the list elements of the object are still in order. We leave the details of these functions as an exercise for you. Furthermore, you can also override the function `seqSearch` so that while performing a sequential search on an ordered list, it takes into account that the elements are in order. We leave the details of this function also as an exercise.

Table 9-4 summarizes the algorithm analysis of the search algorithms discussed earlier.

TABLE 9-4 Number of comparisons for a list of length n

Algorithm	Successful search	Unsuccessful search
Sequential search	$(n + 1) / 2 = O(n)$	$n = O(n)$
Binary search	$2\log_2 n - 3 = O(\log_2 n)$	$2\log_2(n+1) = O(\log_2 n)$

Lower Bound on Comparison-Based Search Algorithms

Sequential and binary search algorithms search the list by comparing the target element with the list elements. For this reason, these algorithms are called **comparison-based search algorithms**. Earlier sections of this chapter showed that a sequential search is of

the order n, and a binary search is of the order $\log_2 n$, where n is the size of the list. The obvious question is: Can we devise a search algorithm that has an order less than $\log_2 n$? Before we answer this question, first we obtain the lower bound on the number of comparisons for the comparison-based search algorithms.

Theorem: Let L be a list of size $n > 1$. Suppose that the elements of L are sorted. If $SRH(n)$ denotes the minimum number of comparisons needed, in the worst case, by using a comparison-based algorithm to recognize whether an element x is in L, then $SRH(n) \geq \log_2(n + 1)$.

Corollary: The binary search algorithm is the optimal worst-case algorithm for solving search problems by the comparison method.

From these results, it follows that if we want to design a search algorithm that is of an order less than $\log_2 n$, it cannot be comparison based.

Hashing

Previous sections of this chapter discussed two search algorithms: sequential and binary. In a binary search, the data must be sorted; in a sequential search, the data does not need to be in any particular order. We also analyzed both these algorithms and showed that a sequential search is of order n, and a binary search is of order $\log_2 n$, where n is the length of the list. The obvious question is: Can we construct a search algorithm that is of order less than $\log_2 n$? Recall that both search algorithms, sequential and binary, are comparison-based algorithms. We obtained a lower bound on comparison-based search algorithms, which shows that comparison-based search algorithms are at least of order $\log_2 n$. Therefore, if we want to construct a search algorithm that is of order less than $\log_2 n$, it cannot be comparison based. This section describes an algorithm that, on average, is of order 1.

The previous section showed that for comparison-based algorithms, a binary search achieves the lower bound. However, a binary search requires the data to be specially organized, that is, the data must be sorted. The search algorithm that we now describe, called **hashing**, also requires the data to be specially organized.

In hashing, the data is organized with the help of a table, called the **hash table**, denoted by ***HT***, and the hash table is stored in an array. To determine whether a particular item with a key, say X, is in the table, we apply a function h, called the **hash function**, to the key X; that is, we compute $h(X)$, read as h of X. The function h is typically an arithmetic function and $h(X)$ gives the address of the item in the hash table. Suppose that the size of the hash table, HT, is m. Then $0 \leq h(X) < m$. Thus, to determine whether the item with key X is in the table, we look at the entry $HT[h(X)]$ in the hash table. Because the address of an item is computed with the help of a function, it follows that the items are stored in no particular order. Before continuing with this discussion, let us consider the following questions:

- How do we choose a hash function?
- How do we organize the data with the help of the hash table?

First, we discuss how to organize the data in the hash table.

There are two ways that data is organized with the help of the hash table. In the first approach, the data is stored within the hash table, that is, in an array. In the second approach, the data is stored in linked lists and the hash table is an array of pointers to those linked lists. Each approach has its own advantages and disadvantages, and we discuss both approaches in detail. However, first we introduce some more terminology that is used in this section.

The hash table HT is, usually, divided into, say b buckets $HT[0]$, $HT[1]$, …, $HT[b-1]$. Each bucket is capable of holding, say r items. Thus, it follows that $br = m$, where m is the size of HT. Generally, $r = 1$ and so each bucket can hold one item.

The hash function h maps the key X onto an integer t, that is, $h(X) = t$, such that $0 \leq h(X) \leq b - 1$.

EXAMPLE 9-2

Suppose there are six students a_1, a_2, a_3, a_4, a_5, a_6 in the Data Structures class and their IDs are a_1: 197354863; a_2: 933185952; a_3: 132489973; a_4: 134152056; a_5: 216500306; and a_6: 106500306.

Let $k_1 = 197354863$, $k_2 = 933185952$, $k_3 = 132489973$, $k_4 = 134152056$, $k_5 = 216500306$, and $k_6 = 106500306$.

Suppose that HT denotes the hash table and HT is of size 13 indexed 0, 1, 2, …, 12.

Define the function h: $\{k_1, k_2, k_3, k_4, k_5, k_6\} \rightarrow \{0, 1, 2, \ldots, 12\}$ by $h(k_i) = k_i \% 13$. (Note that % denotes the mod operator.)

Now

$h(k_1) = h(197354863) = 197354863 \% 13 = 4$	$h(k_4) = h(134152056) = 134152056 \% 13 = 12$
$h(k_2) = h(933185952) = 933185952 \% 13 = 10$	$h(k_5) = h(216500306) = 216500306 \% 13 = 9$
$h(k_3) = h(132489973) = 132489973 \% 13 = 5$	$h(k_6) = h(106500306) = 106500306 \% 13 = 3$

Suppose $HT[b] \leftarrow a$ means "store the data of the student with ID a into $HT[b]$." Then

$HT[4] \leftarrow 197354863$	$HT[5] \leftarrow 132489973$	$HT[9] \leftarrow 216500306$
$HT[10] \leftarrow 933185952$	$HT[12] \leftarrow 134152056$	$HT[3] \leftarrow 106500306$

We consider now a slight variation of Example 9-2.

EXAMPLE 9-3

Suppose there are eight students in the class in a college and their IDs are 197354864, 933185952, 132489973, 134152056, 216500306, 106500306, 216510306, and 197354865. We want to store each student's data into *HT* in this order.

Let $k_1 = 197354864$, $k_2 = 933185952$, $k_3 = 132489973$, $k_4 = 134152056$, $k_5 = 216500306$, $k_6 = 106500306$, $k_7 = 216510306$, and $k_8 = 197354865$.

Suppose that *HT* denotes the hash table and *HT* is of size 13 indexed 0, 1, 2, . . ., 12.

Define the function h: $\{k_1, k_2, k_3, k_4, k_5, k_6, k_7, k_8\} \rightarrow \{0, 1, 2, \ldots, 12\}$ by $h(k_i) = k_i \% 13$. Now

$h(k_1) = 197354864 \% 13 = 5$	$h(k_4) = 134152056 \% 13 = 12$	$h(k_7) = 216510306 \% 13 = 12$
$h(k_2) = 933185952 \% 13 = 10$	$h(k_5) = 216500306 \% 13 = 9$	$h(k_8) = 197354865 \% 13 = 6$
$h(k_3) = 132489973 \% 13 = 5$	$h(k_6) = 106500306 \% 13 = 3$	

As before, suppose $HT[b] \leftarrow a$ means "store the data of the student with ID a into $HT[b]$." Then

$HT[5] \leftarrow 197354864$	$HT[12] \leftarrow 134152056$	$HT[12] \leftarrow 216510306$
$HT[10] \leftarrow 933185952$	$HT[9] \leftarrow 216500306$	$HT[6] \leftarrow 197354865$
$HT[5] \leftarrow 132489973$	$HT[3] \leftarrow 106500306$	

It follows that the data of the student with ID 132489973 is to be stored in $HT[5]$. However, $HT[5]$ is already occupied by the data of the student with ID 197354864. In such a situation, we say that a *collision* has occurred. Later in this section, we discuss some ways to handle collisions.

Two keys, X_1 and X_2, such that $X_1 \neq X_2$, are called **synonyms** if $h(X_1) = h(X_2)$. Let X be a key and $h(X) = t$. If bucket t is full, we say that an **overflow** occurs. Let X_1 and X_2 be two nonidentical keys. If $h(X_1) = h(X_2)$, we say that a **collision** occurs. If $r = 1$, that is, the bucket size is 1, an overflow and a collision occur at the same time.

When choosing a hash function, the main objectives are to:

- Choose a hash function that is easy to compute.
- Minimize the number of collisions.

Next, we consider some examples of hash functions.

Suppose that *HTSize* denotes the size of the hash table, that is, the size of the array holding the hash table. We assume that the bucket size is 1. Thus, each bucket can hold one item and, therefore, overflow and collision occur simultaneously.

Hash Functions: Some Examples

Several hash functions are described in the literature. Here we describe some of the commonly used hash functions.

Mid-Square: In this method, the hash function, h, is computed by squaring the identifier, and then using the appropriate number of bits from the middle of the square to obtain the bucket address. Because the middle bits of a square usually depend on all the characters, it is expected that different keys will yield different hash addresses with high probability, even if some of the characters are the same.

Folding: In folding, the key X is partitioned into parts such that all the parts, except possibly the last parts, are of equal length. The parts are then added, in some convenient way, to obtain the hash address.

Division (Modular arithmetic): In this method, the key X is converted into an integer i_X. This integer is then divided by the size of the hash table to get the remainder, giving the address of X in HT. That is, (in C++)

$h(X) = i_X \% HTSize$;

Suppose that each *key* is a string. The following C++ function uses the division method to compute the address of the key.

```
int hashFunction(char *insertKey, int keyLength)
{
    int sum = 0;

    for (int j = 0; j < keyLength; j++)
        sum = sum + static_cast<int>(insertKey[j]);

    return (sum % HTSize);
} // end hashFunction
```

Collision Resolution

As noted previously, the hash function that we choose not only should be easy to compute, but it is most desirable that the number of collisions is minimized. However, in reality, collisions are unavoidable because usually a hash function always maps a larger domain onto a smaller range. Thus, in hashing, we must include algorithms to handle collisions. Collision resolution techniques are classified into two categories: **open addressing** (also called **closed hashing**), and **chaining** (also called **open hashing**). In open addressing, the data is stored within the hash table. In chaining, the data is organized in linked lists and the hash table is an array of pointers to the linked lists. First we discuss collision resolution by open addressing.

Open Addressing

As described previously, in open addressing, the data is stored within the hash table. Therefore, for each key X, $h(X)$ gives the index in the array where the item with key X is

likely to be stored. Open addressing can be implemented in several ways. Next, we describe some of the common ways to implement it.

LINEAR PROBING

Suppose that an item with key X is to be inserted in HT. We use the hash function to compute the index $h(X)$ of this item in HT. Suppose that $h(X) = t$. Then $0 \leq h(X) \leq HTSize - 1$. If $HT[t]$ is empty, we store this item into this array slot. Suppose that $HT[t]$ is already occupied by another item; we have a collision. In linear probing, starting at location t, we search the array sequentially to find the next available array slot.

In linear probing, we assume that the array is circular so that if the lower portion of the array is full, we can continue the search in the top portion of the array. This can be easily accomplished by using the mod operator. That is, starting at t, we check the array locations t, $(t + 1)$ % $HTSize$, $(t + 2)$ % $HTSize$, ..., $(t + j)$ % $HTSize$. This is called the **probe sequence**.

The next array slot is given by

$(h(X) + j)$ % $HTSize$

where j is the jth probe.

EXAMPLE 9-4

Consider the students' IDs and the hash function given in Example 9-3. Then we know that

h (197354864) = 5 = h (132489973)	h (134152056) = 12 = h (216510306)	h (106500306) = 3
h (933185952) = 10	h (216500306) = 9	h (197354865) = 6

Using the linear probing, the array position where each student's data is stored is:

ID	h(ID)	(h(ID) + 1) % 13	(h(ID) + 2) % 13
197354864	5		
933185952	10		
132489973	5	6	
134152056	12		
216500306	9		
106500306	3		
216510306	12	0	
197354865	6	7	

As before, suppose $HT[b] \leftarrow a$ means "store the data of the student with ID a into $HT[b]$." Then

$HT[5] \leftarrow 197354864$	$HT[12] \leftarrow 134152056$	$HT[0] \leftarrow 216510306$
$HT[10] \leftarrow 933185952$	$HT[9] \leftarrow 216500306$	$HT[7] \leftarrow 197354865$
$HT[6] \leftarrow 132489973$	$HT[3] \leftarrow 106500306$	

The following C++ code implements linear probing:

```
hIndex = hashFunction(insertKey);
found = false;

while (HT[hIndex] != emptyKey && !found)
    if (HT[hIndex].key == key)
        found = true;
    else
        hIndex = (hIndex + 1) % HTSize;

if (found)
    cerr << "Duplicate items are not allowed." << endl;
else
    HT[hIndex] = newItem;
```

From the definition of linear probing, we see that linear probing is easy to implement. However, linear probing causes **clustering**; that is, more and more new keys would likely be hashed to the array slots that are already occupied. For example, consider the hash table of size 20, as shown in Figure 9-5.

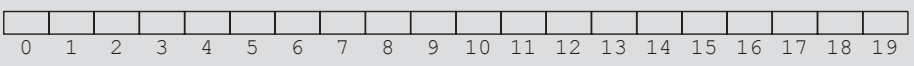

FIGURE 9-5 Hash table of size 20

Initially, all the array positions are available. Because all the array positions are available, the probability of any position being probed is (1/20). Suppose that after storing some of the items, the hash table is as shown in Figure 9-6.

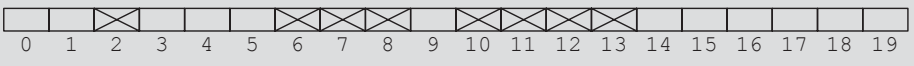

FIGURE 9-6 Hash table of size 20 with certain positions occupied

In Figure 9-6, a cross indicates that this array slot is occupied. Slot 9 will be occupied next if, for the next key, the hash address is 6, 7, 8, or 9. Thus, the probability that slot 9 will be occupied next is 4/20. Similarly, in this hash table, the probability that array position 14 will be occupied next is 5/20.

Now consider the hash table of Figure 9-7.

FIGURE 9-7 Hash table of size 20 with certain positions occupied

In this hash table, the probability that the array position 14 will be occupied next is 9/20, whereas the probability that the array positions 15, 16, or 17 will be occupied next is 1/20. We see that items tend to cluster, which would increase the search length. Linear probing, therefore, causes clustering. This clustering is called **primary clustering**.

One way to improve linear probing is to skip array positions by a fixed constant, say *c*, rather than `1`. In this case, the hash address is as follows:

$(h(X) + i * c) \% HTSize$

If $c = 2$ and $h(X) = 2k$, that is, $h(X)$ is even, only the even-numbered array positions are visited. Similarly, if $c = 2$ and $h(X) = 2k + 1$, that is, $h(X)$ is odd, only the odd-numbered array positions are visited. To visit all the array positions, the constant *c* must be relatively prime to *HTSize*.

9

RANDOM PROBING

This method uses a random number generator to find the next available slot. The *i*th slot in the probe sequence is

$(h(X) + r_i) \% HTSize$

where r_i is the *i*th value in a random permutation of the numbers 1 to $HTSize - 1$. All insertions and searches use the same sequence of random numbers.

EXAMPLE 9-5

Suppose that the size of the hash table is 101, and for the keys X_1 and X_2, $h(X_1) = 26$ and $h(X_2) = 35$. Also suppose that $r_1 = 2$, $r_2 = 5$, and $r_3 = 8$. Then the probe sequence of X_1 has the elements 26, 28, 31, and 34. Similarly, the probe sequence of X_2 has the elements 35, 37, 40, and 43.

REHASHING

In this method, if a collision occurs with the hash function h, we use a series of hash functions, h_1, h_2, ..., h_s. That is, if the collision occurs at $h(X)$, the array slots $h_i(X)$, $1 \leq h_i(X) \leq s$ are examined.

QUADRATIC PROBING

Suppose that an item with key X is hashed at t, that is, $h(X) = t$ and $0 \leq t \leq HTSize - 1$. Further suppose that position t is already occupied. In quadratic probing, starting at position t, we linearly search the array at locations $(t + 1)$ % $HTSize$, $(t + 2^2)$ % $HTSize = (t + 4)$ % $HTSize$, $(t + 3^2)$ % $HTSize = (t + 9)$ % $HTSize$, ..., $(t + i^2)$ % $HTSize$. That is, the probe sequence is: t, $(t + 1)$ % $HTSize$ $(t + 2^2)$ % $HTSize$, $(t + 3^2)$ % $HTSize$, ..., $(t + i^2)$ % $HTSize$.

EXAMPLE 9-6

Suppose that the size of the hash table is 101 and for the keys X_1, X_2, and X_3, $h(X_1) = 25$, $h(X_2) = 96$, and $h(X_3) = 34$. Then the probe sequence for X_1 is 25, 26, 29, 34, 41, and so on. The probe sequence for X_2 is 96, 97, 100, 4, 11, and so on. (Notice that $(96 + 3^2)$ % $101 = 105$ % $101 = 4$.)

The probe sequence for X_3 is 34, 35, 38, 43, 50, 59, and so on. Even though element 34 of the probe sequence of X_3 is the same as the fourth element of the probe sequence of X_1, both probe sequences after 34 are different.

Although quadratic probing reduces primary clustering, we do not know if it probes all the positions in the table. In fact, it does not probe all the positions in the table. However, when *HTSize* is a prime, quadratic probing probes about half the table before repeating the probe sequence. Let us prove this observation.

Suppose that *HTSize* is a prime and for $0 \leq i < j \leq HTSize$,

$$(t + i^2)\%HTSize = (t + j^2)\%HTSize.$$

This implies that *HTSize* divides $(j^2 - i^2)$, that is, *HTSize* divides $(j - i)$ $(j + i)$. Because *HTSize* is a prime, we get *HTSize* divides $(j - i)$ or *HTSize* divides $(j + i)$.

Now because $0 < j - i < HTSize$, it follows that *HTSize* does not divide $(j - i)$. Hence, *HTSize* divides $(j + i)$. This implies that $j + i \geq HTSize$, so $j \geq (HTSize / 2)$.

Hence, quadratic probing probes half the table before repeating the probe sequence. Thus, it follows that if the size of *HTSize* is a prime at least twice the number of items, we can resolve all the collisions.

Because probing half the table is already a considerable number of probes, after making these many probes we assume that the table is full and stop the insertion (and search). (This can occur when the table is actually half full; in practice, it seldom happens unless the table is nearly full.)

Next we describe how to generate the probe sequence.

Note that

$$\begin{aligned} 2^2 &= 1 + (2 \cdot 2 - 1) \\ 3^2 &= 1 + 3 + (2 \cdot 3 - 1) \\ 4^2 &= 1 + 3 + 5 + (2 \cdot 4 - 1) \\ &\vdots \\ i^2 &= 1 + 3 + 5 + 7 + \ldots + (2 \cdot i - 1), \quad i \geq 1. \end{aligned}$$

Thus, it follows that

$$(t + i^2) \,\%\, HTSize = (t + 1 + 3 + 5 + 7 + \ldots + (2 \cdot i - 1)) \,\%\, HTSize$$

Consider the probe sequence t, $t + 1$, $t + 2^2$, $t + 3^2$, ..., $(t + i^2)$ % *HTSize*. The following C++ code computes the ith probe, that is, $(t + i^2)$ % *HTSize*:

```
int inc = 1;
int pCount = 0;

while (p < i)
{
    t = (t + inc) % HTSize;
    inc = inc + 2;
    pCount++;
}
```

The following pseudocode implements quadratic probing (assume that *HTSize* is a prime):

```
int pCount;
int inc;
int hIndex;

hIndex = hashFunction(insertKey);

pCount = 0;
inc = 1;

while (HT[hIndex] is not empty
      && HT[hIndex] is not the same as the insert item
      && pCount < HTSize / 2)
{
    pCount++;
    hIndex = (hIndex + inc ) % HTSize;
    inc = inc + 2;
}

if (HT[hIndex] is empty)
    HT[hIndex] = newItem;
else if (HT[hIndex] is the same as the insert item)
    cerr << "Error: No duplicates are allowed." << endl;
```

```
else
    cerr << "Error: The table is full. "
         << "Unable to resolve the collisions." << endl;
```

Both random and quadratic probings eliminate primary clustering. However, if two nonidentical keys, say X_1 and X_2, are hashed to the same home position, that is, $h(X_1) = h(X_2)$, then the same probe sequence is followed for both keys. The same probe sequence is used for both keys because random probing and quadratic probing are functions of the home positions, not the original key. It follows that if the hash function causes a cluster at a particular home position, the cluster remains under these probings. This is called **secondary clustering**.

One way to solve secondary clustering is to use linear probing, with the increment value a function of the key. This is called **double hashing**. In double hashing, if a collision occurs at $h(X)$, the probe sequence is generated by using the rule:

$(h(X) + i * g(X)) \% HTSize$

where g is the second hash function, and $i = 0, 1, 2, 3, \ldots$.

If the size of the hash table is a prime p, then we can define g as follows:

$g(k) = 1+(k \% (p - 2))$

EXAMPLE 9-7

Suppose that the size of the hash table is 101 and for the keys X_1 and X_2, $h(X_1) = 35$ and $h(X_2) = 83$. Also suppose that $g(X_1) = 3$ and $g(X_2) = 6$. Then the probe sequence for X_1 is 35, 38, 41, 44, 47, and so on. The probe sequence for X_2 is 83, 89, 95, 0, 6, and so on. (Notice that $(83 + 3 * 6) \% 101 = 101 \% 101 = 0$.)

EXAMPLE 9-8

Suppose there are six students in the Data Structures class and their IDs are 115, 153, 586, 206, 985, and 111, respectively. We want to store each student's data in this order. Suppose HT is of the size 19 indexed 0,1,2,3, . . ., 18. Consider the prime number $p = 19$. Then $p - 2 = 17$. For the ID k, we define the hashing functions:

$h(k) = k \% 19$ and $g(k) = 1 + (k \% (p - 2)) = 1 + (k \% 17)$

Let $k = 115$. Now $h(115) = 115 \% 19 = 1$. So the data of the student with ID 115 is stored in $HT[1]$.

Next consider $k = 153$. Now $h(153) = 153 \% 19 = 1$. However, $HT[1]$ is already occupied. So we first calculate $g(153)$, to find the probe sequence of 153. Now $g(153) = 1 + (153 \% 17) = 1 + 0 = 1$. Thus, $h(153) = 1$ and $g(153) = 1$. Therefore, probe sequence of 153 is given by $(h(153) + i \cdot g(153)) \% 19 = (1 + i \cdot 1) \% 19$, $i = 0, 1, 2, 3, \ldots$. Hence,

the probe sequence of 153 is 1, 2, 3, ... Because $HT[2]$ is empty, the data of the student with ID 153 is stored in $HT[2]$.

Consider $k = 586$. Now $h(586) = 586 \% 19 = 16$. Because $HT[16]$ is empty, we store the data of the student with ID 586 in $HT[16]$.

Consider $k = 206$. Now $h(206) = 206 \% 19 = 16$. Because $HT[16]$ is already occupied, we compute $g(206)$. Now $g(206) = 1 + (206 \% 17) = 1 + 2 = 3$. So the probe sequence of 206 is, 16, 0, 3, 6, ... Note that $(16 + 3) \% 19 = 0$. Because $HT[0]$ is empty, the data of the student with ID 206 is stored in $HT[0]$.

We apply this process and find the array position to store the data of each student. If a collision occurs for an ID, then the following table shows the probe sequence of that ID.

ID	h(ID)	g(ID)	Probe sequence	
115	1			
153	1	1	1, 2, 3, 4, 5, . . .	
586	16			
206	16	3	16, 0, 3, 6, 9,	Note that (16 + 3) % 19 = 0
985	16	17	16, 14, 12, 10, . . .	Note that (16 + 17) % 19 = 14
111	16	10	16, 7, 17, 8, . . .	

As before, suppose $HT[b] \leftarrow a$ means "store the data of the student with ID a into $HT[b]$." Then

$HT[1] \leftarrow 115$	$HT[16] \leftarrow 586$	$HT[14] \leftarrow 985$
$HT[2] \leftarrow 153$	$HT[0] \leftarrow 206$	$HT[7] \leftarrow 111$

9

Deletion: Open Addressing

Suppose that an item, say R, is to be deleted from the hash table, HT. Clearly, we first must find the index of R in HT. To find the index of R, we apply the same criteria we applied to R when R was inserted in HT. Let us further assume that after inserting R, another item, R', was inserted in HT, and the home position of R and R' is the same. The probe sequence of R is contained in the probe sequence of R' because R' was inserted in the hash table after R. Suppose that we delete R simply by marking the array slot containing R as empty. If this array position stays empty, then while searching for R' and following its probe sequence, the search terminates at this empty array position. This gives the impression that R' is not in the table, which, of course, is incorrect. The item R cannot be deleted simply by marking its position as empty from the hash table.

One way to solve this problem is to create a special key to be stored in the key of the items to be deleted. The special key in any slot indicates that this array slot is available for

a new item to be inserted. However, during the search, the search should not terminate at this location. This, unfortunately, makes the deletion algorithm slow and complicated.

Another solution is to use another array, say `indexStatusList` of `int`, of the same size as the hash table as follows: Initialize each position of `indexStatusList` to `0`, indicating that the corresponding position in the hash table is empty. When an item is added to the hash table at position, say `i`, we set `indexStatusList[i]` to `1`. When an item is deleted from the hash table at position, say `k`, we set `indexStatusList[k]` to `-1`. Therefore, each entry in the array `indexStatusList` is `-1`, `0`, or `1`.

For example, suppose that you have the hash table as shown in Figure 9-8.

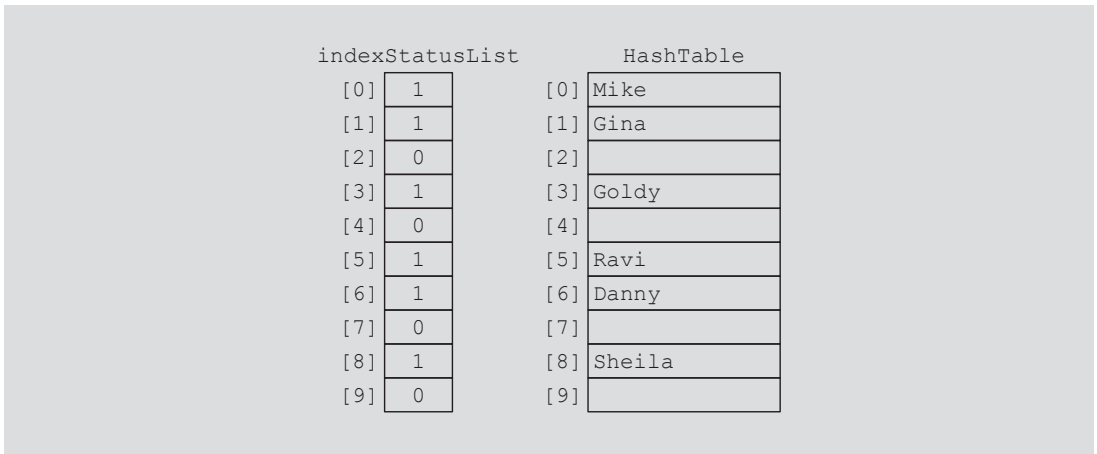

FIGURE 9-8 Hash table and `indexStatusList`

In Figure 9-8, the hash table positions `0`, `1`, `3`, `5`, `6`, and `8` are occupied. Suppose that the entries at positions `3` and `6` are removed. To remove these entries from the hash table, we store `-1` at positions `3` and `6` in the array indexStatusList (see Figure 9-9).

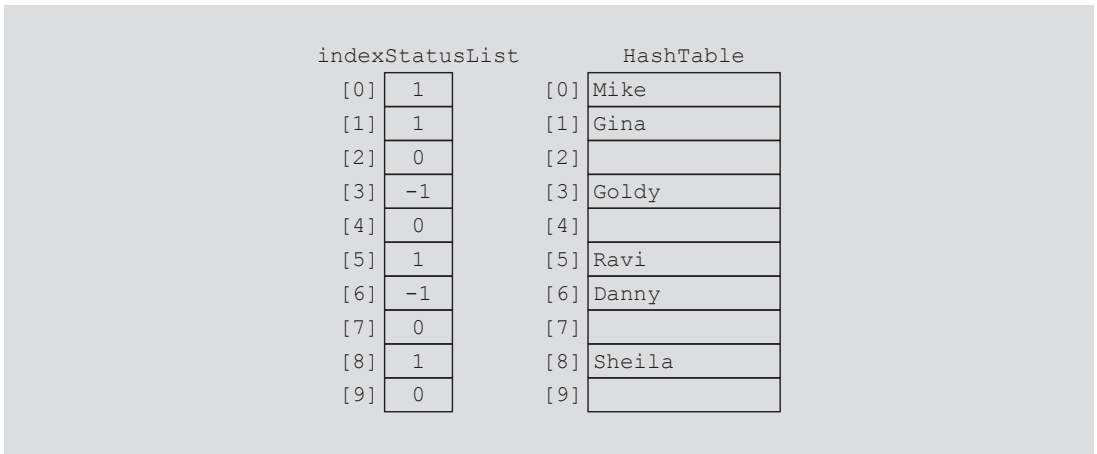

FIGURE 9-9 Hash table and `indexStatusList` after removing the entries at positions `3` and `6`

Hashing: Implementation Using Quadratic Probing

This section briefly describes how to design a class, as an ADT, to implement hashing using quadratic probing. To implement hashing, we use two arrays. One is used to store the data, and the other, `indexStatusList` as described in the previous section, is used to indicate whether a position in the hash table is free, occupied, or used previously. The following `class` template implements hashing as an ADT:

```
//*************************************************************
// Author: D.S. Malik
//
// This class specifies the members to implement a hash table as
// an ADT. It uses quadratic probing to resolve collisions.
//*************************************************************

template <class elemType>
class hashT
{
public:
    void insert(int hashIndex, const elemType& rec);
      //Function to insert an item in the hash table. The first
      //parameter specifies the initial hash index of the item to
      //be inserted. The item to be inserted is specified by the
      //parameter rec.
      //Postcondition: If an empty position is found in the hash
      //   table, rec is inserted and the length is incremented by
      //   one; otherwise, an appropriate error message is
      //   displayed.

    void search(int& hashIndex, const elemType& rec, bool& found) const;
      //Function to determine whether the item specified by the
      //parameter rec is in the hash table. The parameter hashIndex
      //specifies the initial hash index of rec.
      //Postcondition: If rec is found, found is set to true and
      //   hashIndex specifies the position where rec is found;
      //   otherwise, found is set to false.

    bool isItemAtEqual(int hashIndex, const elemType& rec) const;
      //Function to determine whether the item specified by the
      //parameter rec is the same as the item in the hash table
      //at position hashIndex.
      //Postcondition: Returns true if HTable[hashIndex] == rec;
      //   otherwise, returns false.

    void retrieve(int hashIndex, elemType& rec) const;
      //Function to retrieve the item at position hashIndex.
      //Postcondition: If the table has an item at position
      //   hashIndex, it is copied into rec.

    void remove(int hashIndex, const elemType& rec);
      //Function to remove an item from the hash table.
      //Postcondition: Given the initial hashIndex, if rec is found
```

```
      //   in the table it is removed; otherwise, an appropriate
      //   error message is displayed.

    void print() const;
      //Function to output the data.

    hashT(int size = 101);
      //constructor
      //Postcondition: Create the arrays HTTable and indexStatusList;
      //   initialize the array indexStatusList to 0; length = 0;
      //   HTSize = size; and the default array size is 101.

    ~hashT();
      //destructor
      //Postcondition: Array HTable and indexStatusList are deleted.

private:
    elemType *HTable;   //pointer to the hash table
    int *indexStatusList;  //pointer to the array indicating the
                           //status of a position in the hash table
    int length;    //number of items in the hash table
    int HTSize;    //maximum size of the hash table
};
```

We give the definition of only the function `insert` and leave the others as an exercise for you.

The definition of the function `insert` using quadratic probing is as follows:

```
template <class elemType>
void hashT<elemType>::insert(int hashIndex, const elemType& rec)
{
    int pCount;
    int inc;

    pCount = 0;
    inc = 1;

    while (indexStatusList[hashIndex] == 1
           && HTable[hashIndex] != rec && pCount < HTSize / 2)
    {
        pCount++;
        hashIndex = (hashIndex + inc) % HTSize;
        inc = inc + 2;
    }

    if (indexStatusList[hashIndex] != 1)
    {
        HTable[hashIndex] = rec;
        indexStatusList[hashIndex] = 1;
        length++;
    }
```

```
    else if(HTable[hashIndex] == rec)
      cerr << "Error: No duplicates are allowed." << endl;
    else
       cerr << "Error: The table is full. "
            << "Unable to resolve the collision." << endl;
}
```

Chaining

In chaining, the hash table, *HT*, is an array of pointers (see Figure 9-10). Therefore, for each *j*, where $0 \leq j \leq HTSize - 1$, $HT[j]$ is a pointer to a linked list. The size of the hash table, *HTSize*, is less than or equal to the number of items.

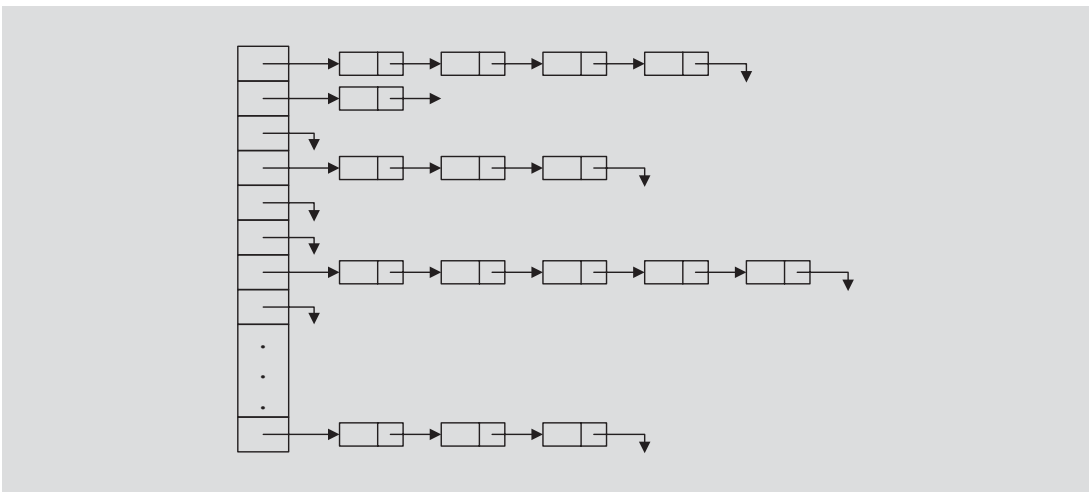

FIGURE 9-10 Linked hash table

ITEM INSERTION AND COLLISION

For each key X (in the item), we first find $h(X) = t$, where $0 \leq t \leq HTSize - 1$. The item with this key is then inserted in the linked list (which might be empty) pointed to by $HT[t]$. It then follows that for nonidentical keys X_1 and X_2, if $h(X_1) = h(X_2)$, the items with keys X_1 and X_2 are inserted in the same linked list and so collision is handled quickly and effectively. (A new item can be inserted at the beginning of the linked list because the data in a linked list is in no particular order.)

SEARCH

Suppose that we want to determine whether an item R with key X is in the hash table. As usual, first we calculate $h(X)$. Suppose $h(X) = t$. Then the linked list pointed to by $HT[t]$ is searched sequentially.

DELETION

To delete an item, say *R*, from the hash table, first we search the hash table to find where in a linked list *R* exists. We then adjust the pointers at the appropriate locations and deallocate the memory occupied by *R*.

OVERFLOW

Because data is stored in linked lists, overflow is no longer a concern because memory space to store the data is allocated dynamically. Furthermore, the size of the hash table no longer needs to be greater than the number of items. If the size of the hash table is less than the number of items, some of the linked lists contain more than one item. However, with a good hash function, the average length of a linked list is still small and so the search is efficient.

ADVANTAGES OF CHAINING

From the construction of the hash table using chaining, we see that item insertion and deletion are straightforward. If the hash function is efficient, few keys are hashed to the same home position. Thus, on average, a linked list is short, which results in a shorter search length. If the item size is large, it saves a considerable amount of space. For example, suppose there are 1000 items and each item requires 10 words of storage. Further suppose that each pointer requires one word of storage. We then need 1000 words for the hash table, 10,000 words for the items, and 1000 words for the link in each node. A total of 12,000 words of storage space, therefore, is required to implement chaining. On the other hand, if we use quadratic probing, if the hash table size is twice the number of items, we need 20,000 words of storage.

DISADVANTAGES OF CHAINING

If the item size is small, a considerable amount of space is wasted. For example, suppose there are 1000 items, each requiring 1 word of storage. Chaining then requires a total of 3000 words of storage. On the other hand, with quadratic probing, if the hash table size is twice the number of items, only 2000 words are required for the hash table. Also, if the table size is three times the number of items, then in quadratic probing the keys are reasonably spread out. This results in fewer collisions and so the search is fast.

Hashing Analysis

Let

$$\alpha = \frac{\text{Number of records in the table}}{HTSize}$$

The parameter α is called the **load factor**.

The average number of comparisons for a successful search and an unsuccessful search are given in Table 9-5.

TABLE 9-5 Number of comparisons in hashing

	Successful search	Unsuccessful search
Linear probing	$\frac{1}{2}\left\{1+\frac{1}{1-\alpha}\right\}$	$\frac{1}{2}\left\{1+\frac{1}{(1-\alpha)^2}\right\}$
Quadratic probing	$\frac{-\log_2(1-\alpha)}{\alpha}$	$\frac{1}{1-\alpha}$
Chaining	$1+\frac{\alpha}{2}$	α

QUICK REVIEW

1. A list is a set of elements of the same type.
2. The length of a list is the number of elements in the list.
3. A one-dimensional array is a convenient place to store and process lists.
4. The sequential search algorithm searches the list for a given item, starting with the first element in the list. It continues to compare the search item with the elements in the list until either the item is found or no more elements are left in the list with which it can be compared.
5. On average, the sequential search algorithm searches half the list.
6. For a list of length n, in a successful search, on average, the sequential search makes $(n + 1) / 2 = O(n)$ comparisons.
7. A sequential search is not efficient for large lists.
8. A binary search is much faster than a sequential search.
9. A binary search requires the list elements to be in order—that is, sorted.
10. For a list of length n, in a successful search, on average, the binary search makes $2 \log_2 n - 3 = O(\log_2 n)$ key comparisons.
11. Let L be a list of size $n > 1$. Suppose that the elements of L are sorted. If SRH(n) is the minimum number of comparisons needed, in the worst case, by using a comparison-based algorithm to recognize whether an element x is in L, then SRH(n) $\geq \log_2(n + 1)$.
12. The binary search algorithm is the optimal worst-case algorithm for solving search problems by using the comparison method.
13. To construct a search algorithm of the order less than $\log_2 n$, it cannot be comparison based.
14. In hashing, the data is organized with the help of a table, called the hash table, denoted by HT. The hash table is stored in an array.